

REST API Design Document (API Specification)

For

Nexora: Multi-Vendor E-Commerce Marketplace

Version: 1.0.0-PROD

Date: July 26, 2026

Prepared By: Office of the Chief API Architect & Enterprise Integration Engineering Group

1. Cover Page

- **Document Title:** Enterprise REST API Specification
- **Project Name:** Nexora Multi-Vendor E-Commerce Marketplace
- **Version:** 1.0.0-PROD
- **Status:** Approved for Implementation

2. Document Information

Attribute	Details
Project Title	Nexora Multi-Vendor E-Commerce Marketplace
Document Type	REST API Design Document & Specification
Target Audience	Enterprise Architecture Board, Core Backend Engineering Teams, Frontend React Engineers, QA Automation Leads, API Security Auditors
Security Classification	Proprietary & Confidential
Primary Technology Stack	Java, Spring Boot, Spring MVC, Spring Security, JWT, MySQL, Redis, Apache Kafka, React, Swagger/OpenAPI

3. Revision History

Date	Version	Author	Description / Change Summary
July 26, 2026	1.0.0	Chief API Architect	Initial production-ready API specification fully aligned with Nexora SRS, System Architecture, and Database Design documents.

4. Table of Contents

1. [Cover Page](#)
2. [Document Information](#)
3. [Revision History](#)
4. [Table of Contents](#)
5. [API Overview](#)
6. [Authentication APIs](#)
7. [Customer APIs](#)
8. [Seller APIs](#)
9. [Administrator APIs](#)
10. [Product APIs](#)
11. [Order APIs](#)
12. [Payment APIs](#)
13. [Notification APIs](#)
14. [Standard Request Format](#)
15. [Standard Success Response](#)
16. [Standard Error Response](#)
17. [Validation Rules](#)
18. [HTTP Status Code Standards](#)
19. [Pagination Standards](#)
20. [Filtering Standards](#)
21. [Security Standards](#)
22. [API Versioning Strategy](#)
23. [Swagger Documentation Standards](#)
24. [Best Practices](#)
25. [Future APIs](#)
26. [Risks and Mitigation](#)
27. [Appendix](#)

5. API Overview

Purpose

This API Design Document defines the complete HTTP/REST interface contract for **Nexora**, a high-performance multi-vendor e-commerce marketplace. It provides exhaustive specifications for backend controllers, request payloads, response payloads, validation rules, security enforcement, and error management, ensuring seamless integration between the React frontend and Spring Boot backend.

Architecture Style

The API strictly implements **REST (Representational State Transfer)** principles, utilizing standard HTTP verbs, stateless communication, resource-based URIs, and consistent JSON representations.

REST Principles

- **Statelessness:** Every request contains all necessary authentication and context data (JWT Bearer Token); server-side sessions are not maintained.
- **Resource-Based URIs:** Endpoints represent collections and individual resources using plural nouns.
- **HTTP Methods:** Proper mapping of verbs (GET, POST, PUT, PATCH, DELETE) to operations.

JSON Standards

All request and response bodies utilize valid UTF-8 encoded JSON. Keys follow camelCase formatting conventions, while ISO-8601 UTC string formats (YYYY-MM-DDTHH:mm:ss.sssZ) represent all timestamps.

API Versioning Strategy

URI path versioning is enforced globally across all endpoints to guarantee backward compatibility and smooth upgrade paths.

- **Base URL Pattern:** /api/v1
- **Example:** https://api.nexora.com/api/v1/products

6. Authentication APIs

6.1 Register Customer

- **HTTP Method:** POST
- **URL:** /api/v1/auth/register/customer
- **Description:** Registers a new customer account on the platform.
- **Authentication Required:** No
- **Authorized Roles:** GUEST
- **Request Parameters:** None

- **Path Variables:** None
- **Query Parameters:** None
- **Request Body Example (JSON):**

JSON

```
{  
  "email" "customer@example.com"  
  "password" "SecurePassword123!"  
  "firstName" "John"  
  "lastName" "Doe"  
  "phoneNumber" "+15550198234"  
}
```

- **Success Response Example (JSON):**

JSON

```
{  
  "success" true  
  "message" "Customer registered successfully. Please verify your email."  
  "data" {  
    "userId" 101  
    "email" "customer@example.com"  
    "role" "ROLE_CUSTOMER"  
    "isEmailVerified" false  
    "createdAt" "2026-07-26T11:18:30.000Z"  
  }  
  "timestamp" "2026-07-26T11:18:30.000Z"  
}
```

- **Error Response Example (JSON):**

JSON

```
{
  "timestamp" "2026-07-26T11:18:30.000Z"
  "status" 400
  "error" "Bad Request"
  "message" "Validation failed for request payload."
  "path" "/api/v1/auth/register/customer"
  "validationErrors"
  "email" "Email address is already registered."
}
```

- **Validation Rules:** Email must be valid and unique; password must be at least 8 characters with upper, lower, number, and special character; first and last names are mandatory.
- **Possible HTTP Status Codes:** 201 Created, 400 Bad Request, 409 Conflict, 500 Internal Server Error

6.2 Register Seller

- **HTTP Method:** POST
- **URL:** /api/v1/auth/register/seller
- **Description:** Registers a new third-party merchant seller account requiring administrative approval.
- **Authentication Required:** No
- **Authorized Roles:** GUEST
- **Request Parameters:** None
- **Path Variables:** None
- **Query Parameters:** None
- **Request Body Example (JSON):**

JSON

```
{
  "email" "seller@store.com"
  "password" "SellerPassword123!"
}
```

```
"storeName" "Apex Electronics"
"storeDescription" "Authorized retailer of high-end consumer electronics."
"taxIdentificationNumber" "GSTIN987654321"
"bankAccountDetails" "ACC-US-998877665544"

```

- Success Response Example (JSON):

```
JSON
{
  "success" true
  "message" "Seller application submitted successfully. Pending administrative approval."
  "data"
    "userId" 102
    "sellerId" 12
    "storeName" "Apex Electronics"
    "applicationStatus" "PENDING"
    "createdAt" "2026-07-26T11:18:30.000Z"
  }
  "timestamp" "2026-07-26T11:18:30.000Z"
}

```

- Error Response Example (JSON):

```
JSON
{
  "timestamp" "2026-07-26T11:18:30.000Z"
  "status" 409
  "error" "Conflict"
  "message" "Store name or Tax ID is already registered."
  "path" "/api/v1/auth/register/seller"
  "validationErrors"
}

```

- **Validation Rules:** Store name, email, and tax ID must be globally unique; bank account details are required and encrypted upon persistence.
- **Possible HTTP Status Codes:** 201 Created, 400 Bad Request, 409 Conflict, 500 Internal Server Error

6.3 Login

- **HTTP Method:** POST
- **URL:** /api/v1/auth/login
- **Description:** Authenticates user credentials and issues a JWT access token and refresh token.
- **Authentication Required:** No
- **Authorized Roles:** GUEST
- **Request Parameters:** None
- **Path Variables:** None
- **Query Parameters:** None
- **Request Body Example (JSON):**

JSON

```
{
  "email" "customer@example.com"
  "password" "SecurePassword123!"
}
```

- **Success Response Example (JSON):**

JSON

```
{
  "success" true
  "message" "Authentication successful."
  "data"
    "accessToken" "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
    "refreshToken" "dGhpcy1pcy1hLXJlZnJlc2gtdG9rZW4tZXhhbXBsZQ=="
}
```

```
"tokenType" "Bearer"
"expiresIn" 900
"roles" "ROLE_CUSTOMER"
"timestamp" "2026-07-26T11:18:30.000Z"
```

- **Error Response Example (JSON):**

```
JSON
{
  "timestamp" "2026-07-26T11:18:30.000Z"
  "status" 401
  "error" "Unauthorized"
  "message" "Invalid email or password credentials."
  "path" "/api/v1/auth/login"
  "validationErrors"
}
```

- **Validation Rules:** Email and password must not be blank; account must not be locked or soft-deleted.
- **Possible HTTP Status Codes:** 200 OK, 401 Unauthorized, 403 Forbidden, 422 Unprocessable Entity, 500 Internal Server Error

6.4 Logout

- **HTTP Method:** POST
- **URL:** /api/v1/auth/logout
- **Description:** Invalidates the current user session and refresh token.
- **Authentication Required:** Yes
- **Authorized Roles:** ROLE_CUSTOMER, ROLE_SELLER, ROLE_ADMIN
- **Request Parameters:** None
- **Path Variables:** None
- **Query Parameters:** None
- **Request Body Example (JSON):**

JSON

{

"refreshToken" "dGhpcy1pcy1hLXJlZnJlc2gtdG9rZW4tZXhhbXBsZQ=="

}

- **Success Response Example (JSON):**

JSON

{

"success" true

"message" "Logged out successfully."

"data" null

"timestamp" "2026-07-26T11:18:30.000Z"

}

- **Error Response Example (JSON):**

JSON

{

"timestamp" "2026-07-26T11:18:30.000Z"

"status" 401

"error" "Unauthorized"

"message" "Invalid or expired JWT token."

"path" "/api/v1/auth/logout"

"validationErrors" []

}

- **Validation Rules:** Bearer token must be present in the Authorization header.

- **Possible HTTP Status Codes:** 200 OK, 401 Unauthorized, 500 Internal Server Error

6.5 Refresh Token

- **HTTP Method:** POST
- **URL:** /api/v1/auth/refresh
- **Description:** Generates a new access token using a valid refresh token.
- **Authentication Required:** No
- **Authorized Roles:** GUEST
- **Request Parameters:** None
- **Path Variables:** None
- **Query Parameters:** None
- **Request Body Example (JSON):**

JSON

```
{  
  "refreshToken" "dGhpcy1pcy1hLXJlZnJlc2gtdG9rZW4tZXhhbXBsZQ=="  
}
```

- **Success Response Example (JSON):**

JSON

```
{  
  "success" true  
  "message" "Token refreshed successfully."  
  "data" {  
    "accessToken" "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9NEW..."  
    "tokenType" "Bearer"  
    "expiresIn" 900  
  }  
  "timestamp" "2026-07-26T11:18:30.000Z"  
}
```

- **Error Response Example (JSON):**

JSON

```
{  
  "timestamp" "2026-07-26T11:18:30.000Z"  
  "status" 401  
  "error" "Unauthorized"  
  "message" "Refresh token is expired or invalid."  
  "path" "/api/v1/auth/refresh"  
  "validationErrors"  
}
```

- **Validation Rules:** Refresh token must exist in storage and not be expired.
- **Possible HTTP Status Codes:** 200 OK, 401 Unauthorized, 500 Internal Server Error

6.6 Forgot Password

- **HTTP Method:** POST
- **URL:** /api/v1/auth/forgot-password
- **Description:** Initiates password reset flow by sending a reset token via email.
- **Authentication Required:** No
- **Authorized Roles:** GUEST
- **Request Parameters:** None
- **Path Variables:** None
- **Query Parameters:** None
- **Request Body Example (JSON):**

JSON

```
{  
  "email" "customer@example.com"  
}
```

- **Success Response Example (JSON):**

JSON

```
{  
  "success" true  
  "message" "Password reset instructions sent to email if account exists."  
  "data" null  
  "timestamp" "2026-07-26T11:18:30.000Z"  
}
```

- **Error Response Example (JSON):**

JSON

```
{  
  "timestamp" "2026-07-26T11:18:30.000Z"  
  "status" 400  
  "error" "Bad Request"  
  "message" "Invalid email format."  
  "path" "/api/v1/auth/forgot-password"  
  "validationErrors"  
    "email" "Email format is invalid."  
}
```

- **Validation Rules:** Email must be valid.
- **Possible HTTP Status Codes:** 200 OK, 400 Bad Request, 500 Internal Server Error

6.7 Reset Password

- **HTTP Method:** POST
- **URL:** /api/v1/auth/reset-password
- **Description:** Resets user password using the token received via email.
- **Authentication Required:** No
- **Authorized Roles:** GUEST
- **Request Parameters:** None
- **Path Variables:** None

- **Query Parameters:** None
- **Request Body Example (JSON):**

JSON

```
{  
  "token" "reset-token-string-xyz"  
  "newPassword" "NewSecurePassword456!"  
}
```

- **Success Response Example (JSON):**

JSON

```
{  
  "success" true  
  "message" "Password has been reset successfully."  
  "data" null  
  "timestamp" "2026-07-26T11:18:30.000Z"  
}
```

- **Error Response Example (JSON):**

JSON

```
{  
  "timestamp" "2026-07-26T11:18:30.000Z"  
  "status" 400  
  "error" "Bad Request"  
  "message" "Password reset token is invalid or expired."  
  "path" "/api/v1/auth/reset-password"  
}
```

```
"validationErrors"
```

- **Validation Rules:** Reset token must be valid; new password must meet complexity standards.
- **Possible HTTP Status Codes:** 200 OK, 400 Bad Request, 500 Internal Server Error

6.8 Verify Email

- **HTTP Method:** GET
- **URL:** /api/v1/auth/verify-email
- **Description:** Verifies user email address via token link.
- **Authentication Required:** No
- **Authorized Roles:** GUEST
- **Request Parameters:** None
- **Path Variables:** None
- **Query Parameters:** token (string, required)
- **Request Body Example (JSON):** None
- **Success Response Example (JSON):**

```
JSON
```

```
"success" true
"message" "Email verified successfully."
"data" null
"timestamp" "2026-07-26T11:18:30.000Z"
```

- **Error Response Example (JSON):**

```
JSON
```

```
"timestamp" "2026-07-26T11:18:30.000Z"
"status" 400
```

```
"error" "Bad Request"
"message" "Verification token is invalid or expired."
"path" "/api/v1/auth/verify-email"
"validationErrors"
```

- **Validation Rules:** Token query parameter must be present and valid.
- **Possible HTTP Status Codes:** 200 OK, 400 Bad Request, 500 Internal Server Error

6.9 Resend Verification

- **HTTP Method:** POST
- **URL:** /api/v1/auth/resend-verification
- **Description:** Resends email verification link to user.
- **Authentication Required:** No
- **Authorized Roles:** GUEST
- **Request Parameters:** None
- **Path Variables:** None
- **Query Parameters:** None
- **Request Body Example (JSON):**

```
JSON
{
  "email" "customer@example.com"
}
```

- **Success Response Example (JSON):**

```
JSON
{
  "success" true
  "message" "Verification email resent successfully."
  "data" null
  "timestamp" "2026-07-26T11:18:30.000Z"
}
```

- **Error Response Example (JSON):**

JSON

```
{
  "timestamp": "2026-07-26T11:18:30.000Z",
  "status": 400,
  "error": "Bad Request",
  "message": "Email is already verified or user does not exist.",
  "path": "/api/v1/auth/resend-verification",
  "validationErrors":
}
```

- **Validation Rules:** Email must be registered and unverified.
- **Possible HTTP Status Codes:** 200 OK, 400 Bad Request, 500 Internal Server Error

6.10 Change Password

- **HTTP Method:** POST
- **URL:** /api/v1/auth/change-password
- **Description:** Updates password for authenticated users.
- **Authentication Required:** Yes
- **Authorized Roles:** ROLE_CUSTOMER, ROLE_SELLER, ROLE_ADMIN
- **Request Parameters:** None
- **Path Variables:** None
- **Query Parameters:** None
- **Request Body Example (JSON):**

JSON

```
{
  "currentPassword": "SecurePassword123!",
  "newPassword": "UpdatedPassword789!"
}
```


- **Success Response Example (JSON):**

```
JSON
{
  "success" true
  "message" "Password changed successfully."
  "data" null
  "timestamp" "2026-07-26T11:18:30.000Z"
}
```

- **Error Response Example (JSON):**

```
JSON
{
  "timestamp" "2026-07-26T11:18:30.000Z"
  "status" 400
  "error" "Bad Request"
  "message" "Current password does not match."
  "path" "/api/v1/auth/change-password"
  "validationErrors"
}
```

- **Validation Rules:** Current password must be correct; new password must meet complexity rules and differ from current password.
- **Possible HTTP Status Codes:** 200 OK, 400 Bad Request, 401 Unauthorized, 500 Internal Server Error

7. Customer APIs

7.1 Profile Management

- **Get Profile** (GET /api/v1/customer/profile)
 - **Auth:** Yes | **Roles:** ROLE_CUSTOMER

- **Success Response (200):** Returns customer profile details (first name, last name, phone number, avatar URL).
- **Update Profile** (PUT /api/v1/customer/profile)
 - **Auth:** Yes | **Roles:** ROLE_CUSTOMER
 - **Request Body:** { "firstName": "John", "lastName": "Smith", "phoneNumber": "+15550198235" }
 - **Success Response (200):** Returns updated profile data.

7.2 Addresses

- **List Addresses** (GET /api/v1/customer/addresses)
 - **Auth:** Yes | **Roles:** ROLE_CUSTOMER
 - **Success Response (200):** Returns list of saved user addresses.
- **Add Address** (POST /api/v1/customer/addresses)
 - **Auth:** Yes | **Roles:** ROLE_CUSTOMER
 - **Request Body:** { "streetAddress": "123 Market St", "city": "Pune", "stateProvince": "Maharashtra", "postalCode": "411001", "country": "India", "isDefault": true }
 - **Success Response (201):** Returns created address object.
- **Update Address** (PUT /api/v1/customer/addresses/{id})
 - **Auth:** Yes | **Roles:** ROLE_CUSTOMER | **Path Variables:** id (long)
 - **Success Response (200):** Returns updated address object.
- **Delete Address** (DELETE /api/v1/customer/addresses/{id})
 - **Auth:** Yes | **Roles:** ROLE_CUSTOMER | **Path Variables:** id (long)
 - **Success Response (204):** No content.

7.3 Wishlist

- **Get Wishlist** (GET /api/v1/customer/wishlist)
 - **Auth:** Yes | **Roles:** ROLE_CUSTOMER
 - **Success Response (200):** Returns customer wishlist items.
- **Add to Wishlist** (POST /api/v1/customer/wishlist/items)
 - **Auth:** Yes | **Roles:** ROLE_CUSTOMER
 - **Request Body:** { "productId": 45 }
 - **Success Response (201):** Returns added wishlist item.
- **Remove from Wishlist** (DELETE /api/v1/customer/wishlist/items/{productId})
 - **Auth:** Yes | **Roles:** ROLE_CUSTOMER | **Path Variables:** productId (long)
 - **Success Response (204):** No content.

7.4 Cart

- **Get Cart** (GET /api/v1/customer/cart)
 - **Auth:** Yes | **Roles:** ROLE_CUSTOMER
 - **Success Response (200):** Returns active cart items and totals.
- **Add Cart Item** (POST /api/v1/customer/cart/items)
 - **Auth:** Yes | **Roles:** ROLE_CUSTOMER

- **Request Body:** { "productVariantId": 102, "quantity": 2 }
- **Success Response (200):** Returns updated cart.
- **Update Cart Item Quantity** (PUT /api/v1/customer/cart/items/{itemId})
 - **Auth:** Yes | **Roles:** ROLE_CUSTOMER | **Path Variables:** itemId (long)
 - **Request Body:** { "quantity": 3 }
 - **Success Response (200):** Returns updated cart.
- **Remove Cart Item** (DELETE /api/v1/customer/cart/items/{itemId})
 - **Auth:** Yes | **Roles:** ROLE_CUSTOMER | **Path Variables:** itemId (long)
 - **Success Response (204):** No content.
- **Clear Cart** (DELETE /api/v1/customer/cart)
 - **Auth:** Yes | **Roles:** ROLE_CUSTOMER
 - **Success Response (204):** No content.

7.5 Orders & Reviews & Notifications

- **Order History** (GET /api/v1/customer/orders) -> Paged order list.
- **Submit Review** (POST /api/v1/customer/reviews) -> { "productId": 45, "rating": 5, "comment": "Excellent product!" }
- **List Notifications** (GET /api/v1/customer/notifications) -> Paged notifications.

8. Seller APIs

- **Seller Profile** (GET / PUT /api/v1/seller/profile) -> Store details & bank info.
- **Store Management** (GET /api/v1/seller/store) -> Storefront metrics.
- **Product Management** (GET / POST / PUT / DELETE /api/v1/seller/products) -> CRUD catalog.
- **Inventory Management** (GET / PUT /api/v1/seller/inventory/{variantId}) -> Stock quantity updates.
- **Order Management** (GET / PUT /api/v1/seller/sub-orders/{id}/status) -> Sub-order fulfillment.
- **Shipment Management** (POST /api/v1/seller/sub-orders/{id}/shipment) -> Tracking number assignment.
- **Dashboard** (GET /api/v1/seller/dashboard/metrics) -> Sales revenue aggregation.

9. Administrator APIs

- **Dashboard** (GET /api/v1/admin/dashboard) -> Platform analytics.
- **User Management** (GET / DELETE /api/v1/admin/users/{id}) -> User governance.
- **Seller Approval** (PUT /api/v1/admin/sellers/{id}/approve) -> Merchant verification.
- **Category Management** (POST / PUT / DELETE /api/v1/admin/categories) -> Taxonomy CRUD.
- **Coupon Management** (POST / GET / DELETE /api/v1/admin/coupons) -> Promo campaigns.
- **Reports** (GET /api/v1/admin/reports/sales) -> Financial audits.
- **Audit Logs** (GET /api/v1/admin/audit-logs) -> Immutable security logs.
- **Platform Configuration** (GET / PUT /api/v1/admin/config) -> System settings.

10. Product APIs

- **Search Products** (GET /api/v1/products/search?query=laptop)
- **Product Details** (GET /api/v1/products/{slug})
- **Featured Products** (GET /api/v1/products/featured)
- **Related Products** (GET /api/v1/products/{id}/related)
- **Latest Products** (GET /api/v1/products/latest)
- **Categories** (GET /api/v1/categories)
- **Filtering & Sorting & Pagination** (Supported across product query parameters).

11. Order APIs

- **Checkout Validation** (POST /api/v1/orders/checkout)
- **Place Order** (POST /api/v1/orders) -> { "shippingAddressId": 1, "billingAddressId": 1, "paymentMethod": "CREDIT_CARD", "couponCode": "SUMMER20" }
- **Cancel Order** (PUT /api/v1/orders/{id}/cancel)
- **Track Order** (GET /api/v1/orders/{id}/tracking)
- **Order History** (GET /api/v1/orders)
- **Invoice Download** (GET /api/v1/orders/{id}/invoice -> PDF stream)

12. Payment APIs

- **Create Payment Intent** (POST /api/v1/payments/intent)
- **Verify Payment** (POST /api/v1/payments/verify)
- **Payment Status** (GET /api/v1/payments/{orderId})
- **Process Refund** (POST /api/v1/payments/{id}/refund)
- **Payment History** (GET /api/v1/payments)

13. Notification APIs

- **List Notifications** (GET /api/v1/notifications)
- **Mark Read** (PUT /api/v1/notifications/{id}/read)
- **Delete Notification** (DELETE /api/v1/notifications/{id})

14. Standard Request Format

Standard HTTP headers:

- Content-Type: application/json
- Accept: application/json
- Authorization: Bearer <JWT_ACCESS_TOKEN>

15. Standard Success Response

JSON

```
|  
|  
| "success" true  
| "message" "Operation executed successfully."  
| "data"  
| "timestamp" "2026-07-26T11:18:30.000Z"  
|
```

16. Standard Error Response

```
JSON  
|  
| "timestamp" "2026-07-26T11:18:30.000Z"  
| "status" 400  
| "error" "Bad Request"  
| "message" "Validation failed for request payload."  
| "path" "/api/v1/endpoint"  
| "validationErrors"  
| "fieldName" "Constraint violation message."  
|
```

17. Validation Rules

- **Bean Validation:** Enforced via Spring @Valid annotations (@NotNull, @Size, @Email, @Min, @Max).
- **Input Validation:** Sanitization against XSS and SQL injection at controller boundaries.
- **Business Validation:** Service-layer validation enforcing database rules (e.g., sufficient stock, unique SKUs, active seller status).

18. HTTP Status Code Standards

- 200 OK: Successful request.
- 201 Created: Resource successfully created.
- 204 No Content: Successful action with no response body.
- 400 Bad Request: Malformed syntax or validation failure.
- 401 Unauthorized: Missing or invalid authentication token.
- 403 Forbidden: Insufficient role privileges.
- 404 Not Found: Target resource does not exist.
- 409 Conflict: Resource state conflict (e.g., duplicate email).

- 422 Unprocessable Entity: Business logic validation failure.
- 500 Internal Server Error: Unhandled server exception.

19. Pagination Standards

Query parameters for paged endpoints:

- page: Zero-based page index (default: 0)
- size: Number of records per page (default: 20, max: 100)
- sort: Field name to sort by (e.g., price, createdAt)
- direction: Sort order (ASC or DESC)

20. Filtering Standards

Query parameters for filtering collections:

- category: Category slug or ID
- brand: Brand name string
- minPrice / maxPrice: Numeric price range
- rating: Minimum star rating filter
- availability: Boolean stock filter
- seller: Seller ID or store slug

21. Security Standards

- **JWT:** Signed HMAC-SHA256 tokens containing user claims and roles.
- **Bearer Token:** Passed via standard Authorization header.
- **Role-Based Access:** Enforced via Spring Security @PreAuthorize("hasRole('ADMIN')").
- **Password Encryption:** Bcrypt hashing with strength 12.
- **Rate Limiting:** IP and user-based throttling via Redis buckets.
- **CORS:** Configured explicitly for approved React frontend client origins.

22. API Versioning Strategy

- **Current Version:** v1 (/api/v1/...)
- **Future Version:** v2 reserved for breaking schema changes.
- **Backward Compatibility:** Maintained through deprecation headers (Sunset and Deprecation) prior to endpoint retirement.

23. Swagger Documentation Standards

- OpenAPI 3.0 annotations (@Operation, @ApiResponse, @Parameter) mandatory across all controllers.
- DTO fields documented with @Schema(description = "...", example = "...").
- Standard error response schemas globally registered in OpenAPI components.

24. Best Practices

- Resource-based plural naming conventions (/products, /orders).
- Proper usage of HTTP verbs (GET for retrieval, POST for creation, PUT for full replacement, PATCH for partial updates, DELETE for removal).
- Idempotency ensured for payment and order placement endpoints via unique request tracking keys.

- Caching headers (Cache-Control, ETag) implemented for static catalog assets.

25. Future APIs

1. Real-time customer support chat endpoints (/api/v1/chat/...)
2. Automated product return request endpoints (/api/v1/returns/...)
3. Refund workflow tracking endpoints (/api/v1/refunds/...)
4. Loyalty reward point balance & redemption endpoints (/api/v1/loyalty/...)
5. AI-powered personalized recommendation endpoints (/api/v1/recommendations/...)
6. Advanced sales analytics & BI report export endpoints (/api/v1/analytics/...)
7. WebSocket endpoints for live order tracking (/ws/orders/...)
8. Live inventory stock streaming endpoints (/ws/inventory/...)
9. Semantic AI product search endpoints (/api/v1/search/ai/...)
10. Voice-activated search query endpoints (/api/v1/search/voice/...)
11. Multi-currency conversion rate endpoints (/api/v1/currency/...)
12. B2B bulk quotation negotiation endpoints (/api/v1/b2b/quotes/...)
13. Augmented reality 3D model asset endpoints (/api/v1/products/{id}/ar-model)
14. Gift card purchase and redemption endpoints (/api/v1/gift-cards/...)
15. Flash sale countdown and reservation endpoints (/api/v1/flash-sales/...)
16. Vendor subscription plan billing endpoints (/api/v1/seller/subscriptions/...)
17. Multi-warehouse inventory routing endpoints (/api/v1/warehouses/...)
18. Customer wishlist sharing endpoints (/api/v1/customer/wishlist/share)
19. Advanced product bundle & kit pricing endpoints (/api/v1/bundles/...)
20. Automated tax calculation webhooks (/api/v1/taxes/calculate)
21. Freight carrier rate quotation endpoints (/api/v1/shipping/rates)
22. Automated supplier dropshipping sync endpoints (/api/v1/dropship/...)
23. Customer product question & answer endpoints (/api/v1/products/{id}/qa)
24. Vendor performance rating ledger endpoints (/api/v1/seller/ratings/...)
25. Automated fraud scoring check endpoints (/api/v1/security/fraud-check)
26. Marketing email campaign management endpoints (/api/v1/admin/marketing/...)
27. Dynamic pricing rule engine endpoints (/api/v1/admin/pricing-rules/...)
28. Push notification device token registration endpoints (/api/v1/devices/...)
29. Data privacy GDPR compliance export endpoints (/api/v1/customer/privacy/export)
30. System health check and readiness probe endpoints (/actuator/health, /actuator/metrics)

26. Risks and Mitigation

Risk Identified	Impact	Probability	Mitigation Strategy
API Token Compromise / Session Hijacking	High	Medium	Implement short-lived JWT access tokens (15 mins), secure

			refresh tokens, and token revocation lists in Redis.
High Traffic DDoS on Product Search	High	Medium	Implement Redis-based rate limiting, response caching, and CDN edge caching for catalog endpoints.
Payload Validation Bypass & Injection	High	Low	Enforce strict Bean Validation (@Valid) and Spring Security parameterized query sanitization.
Breaking Changes for Frontend Clients	Medium	Medium	Enforce strict URI path versioning (/api/v1) and deprecation policy lifecycles.

27. Appendix

Appendix A: Reference Standards

- OpenAPI Specification 3.0.3.
- RESTful API Modeling Guide (Richardson Maturity Level 3).
- RFC 7807 Problem Details for HTTP APIs.